NỘI DUNG TRÌNH BÀY

- Tạo các applet
- Đối tượng đồ hoạ Graphics
- Kĩ thuật khung hình phụ

PHẦN 1
TẠO CÁC APPLET

XÂY DỰNG CÁC APPLET

- Lớp Applet
  - Java có lớp **java.applet.Applet** kế thừa từ lớp java.awt.Component cho phép tạo ra các applet trong Web.
  - Mọi lớp applet do người dùng tạo ra đều phải kế thừa từ lớp Applet.

XÂY DỰNG CÁC APPLET

Lớp Applet:

- **Chạy trên trình duyệt**: Applet chạy trên máy tính của người dùng thông qua trình duyệt web.
- **Sử dụng Java API**: Nó có thể dùng toàn bộ các thư viện của Java để tạo giao diện hoặc xử lý dữ liệu.
- **Bảo mật**: Do chạy trên trình duyệt, applet bị giới hạn quyền truy cập vào hệ thống để đảm bảo an toàn

GV: Võ Tấn Dũng

XÂY DỰNG CÁC APPLET

Vòng đời của một applet gồm 5 sự kiện chính:

- **init**: Khởi tạo chương trình khi được tải lần đầu.
- **start**: Bắt đầu chạy applet.
- **paint**: Vẽ và hiển thị giao diện lên màn hình.
- **stop**: Tạm dừng applet khi người dùng rời khỏi trang web.
- **destroy**: Hủy applet khi không còn sử dụng để giải phóng bộ nhớ.

XÂY DỰNG CÁC APPLET

- Công nghệ này hiện đã lỗi thời.
- Các trình duyệt hiện đại đã ngừng hỗ trợ plugin chạy applet từ lâu.
- Nền tảng Java cũng đã chính thức loại bỏ hoàn toàn applet như thông báo từ Inside.java.
- Tìm hiểu thêm về lý thuyết lập trình và còn một số trang web cũ (chưa nâng cấp) vẫn còn hoạt động.
- Nay: Spring Framework (đặc biệt là Spring Boot): chuẩn và phổ thông
  - Và một số công nghệ khác (phức tạp hơn)
  - ….

XÂY DỰNG CÁC APPLET

- Ví dụ 1: Tạo file TestApplet.java

```
import java.applet.Applet;
import java.awt.Graphics;
public class TestApplet extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("Helloworld!", 50, 25);
    }
}
```

- Dịch: javac TestApplet.java

XÂY DỰNG CÁC APPLET

- Thực thi applet
  - ***Cách 1:*** Tạo file TestApplet.html có nội dung như sau:

```
<APPLET CODE="TestApplet.class" WIDTH=500 HEIGHT=500
</APPLET>
```

  - Mở file này bằng trình duyệt WEB
  - ***Cách 2:*** Dùng công cụ appletviewer.
  - Gõ lệnh:

appletviewerTestApplet.htmlT>

GV: Võ Tấn Dũng

KHUNG CỦA MỘT APPLET CƠ BẢN

```
import java.applet.Applet;
import java.awt.Graphics;
public class TestApplet extends Applet
{
    public void init() {…}
    public void start() {…}
    public void stop() {…}
    public void destroy {…}
    public void paint(Graphics g) {…}
}
```


GV: Võ Tấn Dũng

HOẠT ĐỘNG CỦA APPLET

- init(): khởi tạo applet
- start(): applet bắt đầu hoạt động
- stop(): applet chấm dứt hoạt động
- destroy(): giải phóng applet
- Chú ý:
  - paint() không phải là phương thức của Applet mà là của Component.
  - paint() được gọi mỗi khi cửa sổ được vẽ lại.

HOẠT ĐỘNG CỦA MỘT APPLET

Vòng đời của một Applet

- Nạp một applet: applet được khởi tạo và thực thi
- Chuyển hoặc trở về trang Web: Các phương thức stop và start sẽ được gọi
- Nạp lại applet: như quá trình nạp applet
- Thoát khỏi trình duyệt: phương thức stop và destroy sẽ được gọi

GV: Võ Tấn Dũng

MỘT SỐ METHOD CỦA COMPONENT

- void setVisible(boolean): hiển thị hoặc ẩn component
- Dimension getSize(): trả về kích thước của component
- void setSize(Dimension): thay đổi kích thước
- void setEnabled(): "bật" hoặc "tắt" component
- void repaint(): cập nhật lại component
- void update (Graphics g): được gọi qua repaint()
- void paint (Graphics g): được gọi qua update()
- void setBackground(Color): đặt màu nền
- ...

GV: Võ Tấn Dũng

PHẦN 2
LỚP GRAPHICS

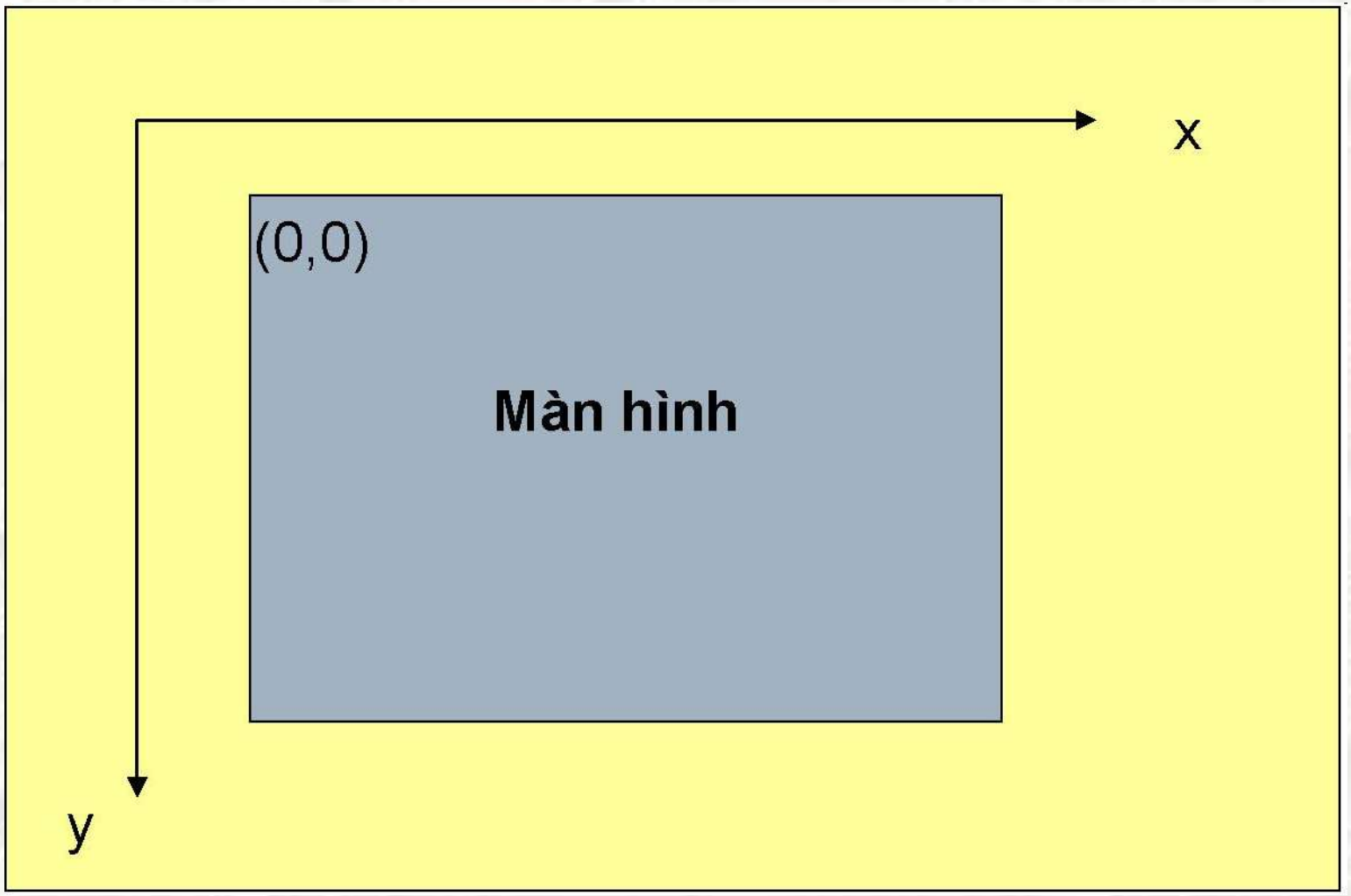
LỚP GRAPHICS

- java.awt.Graphics là lớp cung cấp các phương thức vẽ đồ hoạ cơ bản:
  - Đường thẳng (Line)
  - Đường oval (Oval)
  - Hình chữ nhật (Rectangle)
  - Đa giác (Polygon)
  - Văn bản(Text)
  - Hình ảnh (Image)
  - ...

GV: Võ Tấn Dũng

LỚP GRAPHICS

- Hệ tọa độ



LỚP GRAPHICS

- Vẽ đường thẳng
  - public void drawLine(int x1, int y1, int x2, int y2);
- Vẽ hình chữ nhật
  - public void drawRect(int x, int y, int width, int height);
- Tô một hình chữ nhật
  - public void fillRect(int x, int y, int width, int height);
- Xoá một vùng chữ nhật
  - public void clearRect(int x, int y, int width, int height);
- Vẽ đa giác
  - public void drawPolygon(int[] x, int[] y, int numPoint);
  - public void drawPolygon(Polygon p);

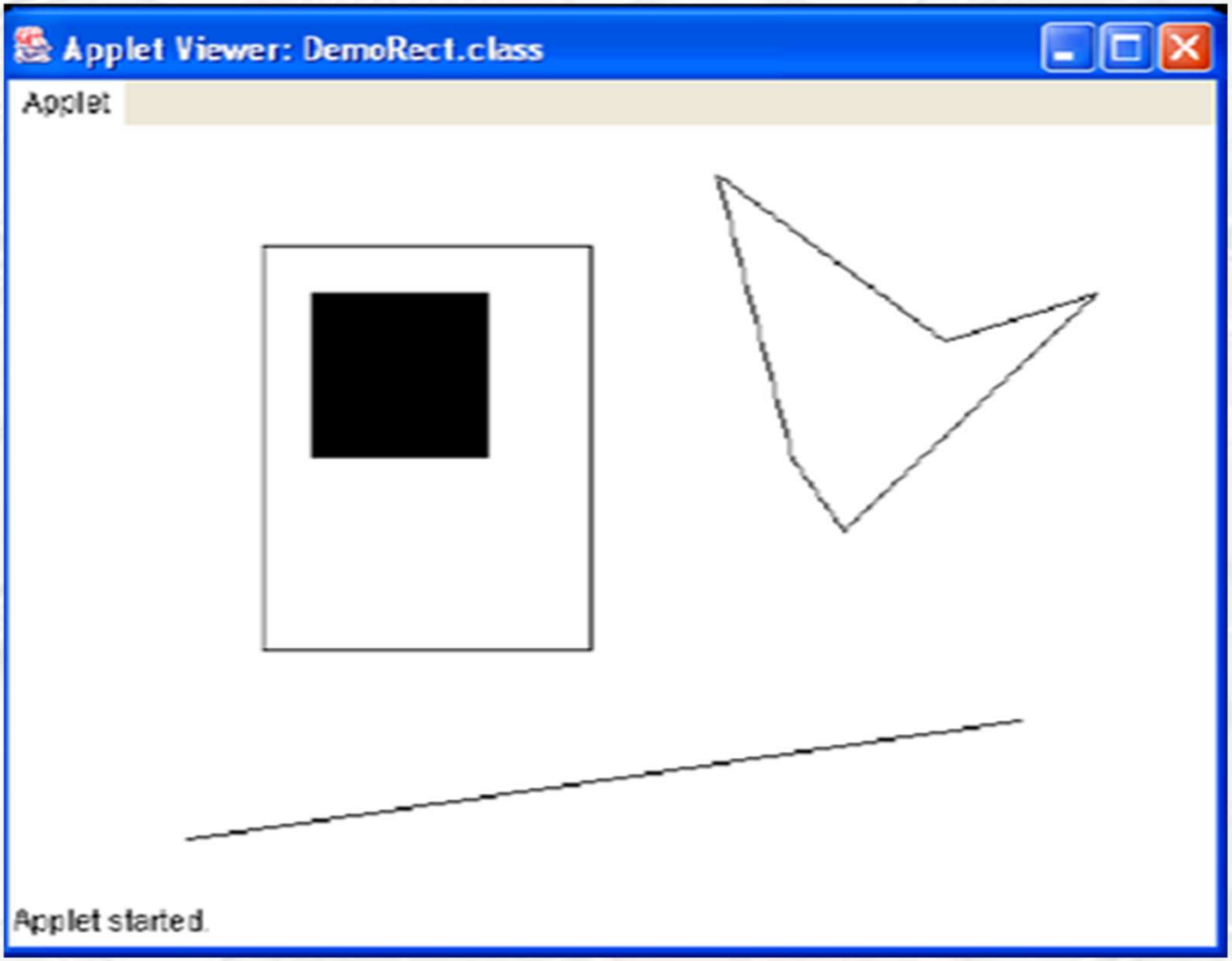
x1, y1: bắt đầu
x2, y2: kết thúc

LỚP GRAPHICS

```
import java.applet.Applet;
import java.awt.Graphics;
public class DemoRect extends Applet{
    public void init()   {
        System.out.println("Demonstration of some simple figures");
    }
    public void paint(Graphics g)    {
        g.drawLine(70, 300, 400, 250);
        g.drawRect(100, 50, 130, 170);
        g.fillRect(120, 70, 70, 70);
        int[] x = { 280, 310, 330, 430, 370 };
        int[] y = { 280, 140, 170, 70, 90 };
        g.drawPolygon(x, y, x.length);
    }
```


GV: Võ Tấn Dũng

LỚP GRAPHICS



LỚP GRAPHICS

- Vẽ đường tròn/elip
  - public void drawOval(int x, int y, int width, int height);
- Tô đường tròn/elip
  - public void fillOval(int x, int y, int width, int height);
- Vẽ cung tròn
  - public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle);
- Vẽ xâu kí tự
  - public void drawString(String str, int x, int y);
- Vẽ ảnh
  - public void drawImage(Image img, int x, int y,...);

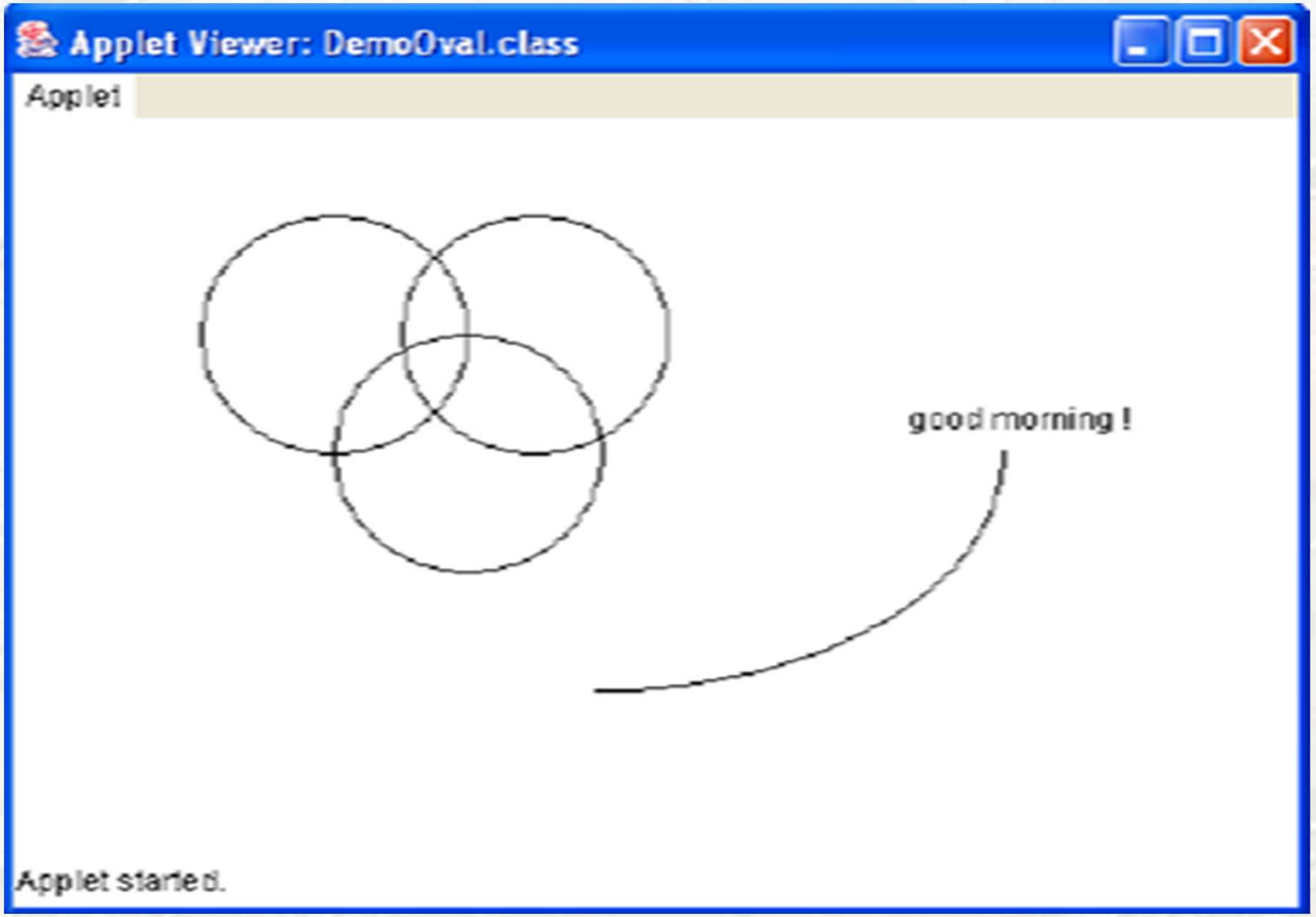
GV: Võ Tấn Dũng

LỚP GRAPHICS

```
import java.applet.Applet;
import java.awt.Graphics;
public class DemoOval extends Applet{
    public void init()        {
            System.out.println("Demonstration of some simple figures");
    }
    public void paint(Graphics g) {
            int xstart = 70, ystart = 40, size = 100;
            g.drawOval(xstart, ystart, size, size);
            g.drawOval(xstart + (size*3)/4, ystart, size, size);
            g.drawOval(xstart + size/2, ystart + size/2, size, size);
            g.drawArc(xstart, ystart, 300, 200, 0, -90);
            g.drawString("good morning !", xstart + 265, ystart + 90);
    }
}
```


GV: Võ Tấn Dũng

LỚP GRAPHICS



LỚP GRAPHICS

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Image;
public classDemoImage extends Applet{
    public void init()   {
        System.out.println("Demonstration of imaging");
    }
    public void paint(Graphics g)    {
        Image image = getToolkit().getImage("summer.jpg");
        g.drawImage(image, 0, 0, this);
    }
}
```


GV: Võ Tấn Dũng

LỚP GRAPHICS



CÁC LỚP TIỆN ÍCH KHÁC

- ***LớpPoint:*** biểu diễn điểm trên màn hình
- ***Lớp Dimension:*** biểu diễn kích thước về chiều rộng và chiều cao của một đối tượng
- ***Lớp Rectangle:*** biểu diễn hình chữ nhật
- ***Lớp Polygon:*** biểu diễn đa giác
- ***Lớp Color:*** biểu diễn màu sắc

GV: Võ Tấn Dũng

CÁC LỚP TIỆN ÍCH KHÁC

```
import java.applet.Applet;
import java.awt.*;
public class DemoColor extends Applet{
    public void paint(Graphics g)      {
        Dimension size = getSize();
        g.setColor(Color.orange);
        g.fillRect(0, 0, size.width, size.height);
        Color color = new Color(10, 150, 20);
        g.setColor(color);
        g.drawString("I am a colorful string",
                        size.width/2 -50, size.height/2);
    }
```


GV: Võ Tấn Dũng

CÁC LỚP TIỆN ÍCH KHÁC

- Máy tính biểu diễn màu RGB bằng cách kết hợp ba cường độ ánh sáng cơ bản gồm Đỏ (Red), Xanh lá (Green) và Xanh dương (Blue)
- **Nguyên lý hoạt động:**
- **Hệ màu cộng:** Ánh sáng từ ba màu được cộng lại với nhau để tạo ra các màu sắc khác.
- **Màn hình tối:** Màn hình bắt đầu từ màu đen (không có ánh sáng) rồi thêm ánh sáng vào.
- **Điểm ảnh (Pixel):** Mỗi điểm ảnh trên màn hình gồm ba điểm sáng nhỏ (sub-pixel) ứng với ba màu đỏ, xanh lá và xanh dương

GV: Võ Tấn Dũng

CÁC LỚP TIỆN ÍCH KHÁC

Cách mã hóa dữ liệu

- **Mỗi kênh màu:** Sử dụng **8 bit** (1 byte) bộ nhớ.
- **Dải giá trị:** Mỗi kênh có giá trị số nguyên từ **0 đến 255** (tổng cộng 256 mức độ sáng).
- **Tổng số màu:** Tạo ra 256 × 256 × 256 ≈ 16,7 triệu màu (True Color)

GV: Võ Tấn Dũng

CÁC LỚP TIỆN ÍCH KHÁC

Ví dụ cụ thể:

- **Màu đen:** (0, 0, 0) – Không bật màu nào.
- **Màu trắng:** (255, 255, 255) – Cả ba màu đạt độ sáng tối đa.
- **Màu đỏ thuần:** (255, 0, 0) – Chỉ bật màu đỏ ở mức cao nhất.
- **Màu xanh lá thuần:** (0, 255, 0) – Chỉ bật màu xanh lá.
- **Màu xanh biển:** (0, 0, 255) – Chỉ bật màu xanh biển.

GV: Võ Tấn Dũng

CÁC LỚP TIỆN ÍCH KHÁC



XỬ LÝ FONT VẼ

```
import java.applet.Applet;
import java.awt.*;
public class DemoFont extends Applet{
    public void paint(Graphics g)    {
        Font font = newFont("Arial", Font.BOLD, 30);
        g.setFont(font);
        g.drawString("I am font Arial, bold, size 30", 50, 50);
    }
}
```



PHẦN 3
KỸ THUẬT KHUNG HÌNH PHỤ

GV: Võ Tấn Dũng

KỸ THUẬT KHUNG HÌNH PHỤ

- Ví dụ về sự di chuyển "bị nháy"

```
import java.applet.Applet;
import java.awt.*;
public class DemoMove1 extends Applet{
    private int x = 50;
    private int y = 50;
    public void paint(Graphics g)    {
        if(x > 300) x = 50;
        g.fillOval(x, y, 100, 100);
        delay(100);
        move();
    }
```

//xem tiếp ở slide tiếp theo

GV: Võ Tấn Dũng

KỸ THUẬT KHUNG HÌNH PHỤ

- Ví dụ về sự di chuyển "bị nháy"

```
public void delay(int milisecond) {
        try{
                Thread.sleep(milisecond);
        } catch(Exception e) { }
}
public void move()    {
        x += 5;
        repaint();
        // ve lai cua so
    }
}
```


GV: Võ Tấn Dũng

KỸ THUẬT KHUNG HÌNH PHỤ

- Giải thích lý do nháy:
  - Mỗi lần gọi repaint() thì update() được gọi
  - update() xoá nội dung cửa sổvà gọi paint()
- Khắc phục:
  - Vẽ hình vào một khung hình phụ trong bộ nhớ thay vì vẽ trực tiếp ra màn hình.
  - Khi vẽ xong, khung hình phụ được hiển thị ra màn hình trong phương thức paint().
  - Nạp chồng update() để tránh việc xoá nội dung cửa sổ.

GV: Võ Tấn Dũng

KỸ THUẬT KHUNG HÌNH PHỤ

- Giải pháp

```
//...
public class DemoMove2 extends Applet
{
    //...
    private Image offImage;
    private Graphics offGraphics;
    public void init()
    {
            // tao khung hinh phu
            offImage = createImage(500, 500);
            // lay doi tuong do hoa de ve vao khung hinh phu
            offGraphics = offImage.getGraphics();
    }
//xem tiếp ở slide tiếp theo
```


GV: Võ Tấn Dũng

KỸ THUẬT KHUNG HÌNH PHỤ

- Giải pháp

```
public void paint(Graphics g)
{
        //...
        offGraphics.clearRect(0, 0, 500, 500);
        offGraphics.fillOval(x, y, 100, 100);
        g.drawImage(offImage, 0, 0, this);
        //...
}
public voidu pdate(Graphics g)
{
        paint(g);
}
//...
}
```


GV: Võ Tấn Dũng

CLIENT / SERVER

Client/Server (Máy khách/Máy chủ):

- Trong lập trình web là kiến trúc phân tách rõ ràng giữa bên gửi yêu cầu (Client) và bên xử lý, cung cấp dịch vụ (Server) qua giao thức mạng như HTTP/HTTPS
- **Client (Máy khách):** Là trình duyệt web (như Chrome, Firefox, Safari) hoặc ứng dụng trên thiết bị người dùng.
  - **Nhiệm vụ:** Gửi yêu cầu (Request) và hiển thị giao diện, xử lý logic phía người dùng (Client-side) bằng HTML, CSS, JavaScript.

GV: Võ Tấn Dũng

CLIENT / SERVER

Client/Server (Máy khách/Máy chủ):

- **Server (Máy chủ):** Là hệ thống máy tính hoặc phần mềm chạy trên máy chủ (như Nginx, Apache, Node.js, Python/Django).
  - **Nhiệm vụ:** Nhận yêu cầu từ Client, xử lý logic nghiệp vụ (Business Logic), truy vấn cơ sở dữ liệu (Database) và trả về kết quả (Response)

GV: Võ Tấn Dũng

CLIENT / SERVER

Nguyên lý hoạt động:

- **Gửi yêu cầu (Request):** Người dùng nhập địa chỉ trang web hoặc thao tác trên giao diện, [Client] gửi một HTTP/HTTPS Request đến [Server].
- **Xử lý (Processing):** [Server] tiếp nhận yêu cầu, đọc dữ liệu, tính toán, kết nối cơ sở dữ liệu nếu cần.
- **Phản hồi (Response):** [Server] trả dữ liệu về cho [Client] (thường là mã HTML/CSS/JS hoặc dữ liệu dạng JSON/XML).
- **Hiển thị (Display):** [Client] nhận dữ liệu và render (hiển thị) thành giao diện hoàn chỉnh cho người dùng xem

GV: Võ Tấn Dũng

CLIENT / SERVER

- **Ưu điểm: Quản lý tập trung:** Dữ liệu và mã nguồn chính được lưu trên Server, giúp bảo mật và dễ đồng bộ.
- **Dễ mở rộng:** Có thể nâng cấp phần cứng hoặc tối ưu Server riêng biệt mà không ảnh hưởng trực tiếp đến Client.

GV: Võ Tấn Dũng

CLIENT / SERVER

- **Nhược điểm: Phụ thuộc vào mạng:** Cần có kết nối Internet hoặc mạng LAN ổn định.
- **Quá tải:** Khi quá nhiều Client cùng truy cập một lúc, Server có nguy cơ bị chậm hoặc sập nếu không đủ tài nguyên

GV: Võ Tấn Dũng

3-layer và 3-tiers

- Phân biệt giữa 3-layer và 3-tiers.
- Mô hình Hướng đối tượng 3 lớp trong công nghệ lập trình
- Lợi thế của mô hình 3 lớp
- Chi tiết về mô hình 3 lớp
- Ví dụ minh họa

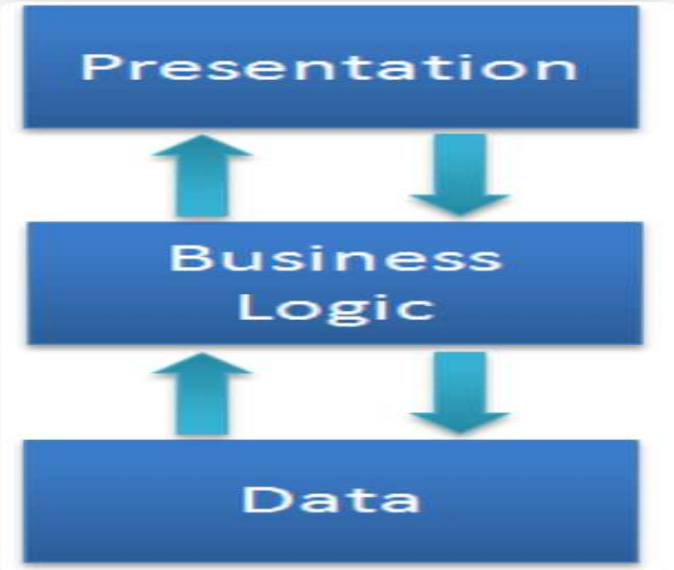
3-layer và 3-tiers

- Mô hình **3 tầng** (**3-tiers** architecture) là gì?
- Mô hình **3 lớp** (**3-layers** architecture) là gì?
- Sự khác biệt giữa 3-layer và 3-tiers?

➔ Chúng có ý nghĩa như thế nào và tại sao ta phải dùng mô hình 3-layers để viết theo mô hình này trong ứng dụng quản lý dữ liệu vận dụng công nghệ lập trình?

3-tiers

- **3-Tiers** có tính vật lý (physical): là mô hình client-server (mỗi tier có thể đặt chung 1 nơi hoặc nhiều nơi, kết nối với nhau qua Web services, WCF, Remoting...).
- 3 tầng rõ rệt:



3-tiers

Mô hình này phân tách ứng dụng ra làm 3 module riêng biệt, bao gồm:

– Tầng Presentation: được dùng để giao tiếp với người dùng, nhiệm vụ chính là hiển thị dữ liệu và nhận dữ liệu từ người dùng.

– Tầng Business Logic: nhiệm vụ chính là cung cấp các chức năng của phần mềm.

– Tầng Data: lưu trữ dữ liệu, cho phép lớp Business Logic có thể tìm kiếm, trích xuất, cập nhật… dữ liệu.

3-tiers

- **Ưu điểm:**
  - Dễ dàng mở rộng, thay đổi quy mô của hệ thống: Khi cần tải lớn, người quản trị có thể dễ dàng thêm các máy chủ vào nhóm, hoặc lấy bớt ra trong trường hợp ngược lại.
  - Khi ứng dụng đòi hỏi có sự tách biệt 3 phần: giao diện, xử lý nghiệp vụ, giao tiếp với hệ quản trị CSDL để người viết có thể dễ dàng quản lý ứng dụng của mình khi có bug xảy ra với 1 số thành phần xử lý nghiệp vụ không mong muốn. Ngoài ra với mô hình này nó còn tạo ra 1 không gian làm việc rất tốt để người thiết kế giao diện, lẫn người lập trình có thể làm việc chung với nhau 1 cách dễ dàng.
  - Khả năng tái tạo cao: Khi ứng dụng bất chợt yêu cầu thay đổi hệ quản trị CSDL hoặc chuyển ứng dụng từ window application sang web application, việc xây dựng lại ứng dụng từ đầu rất tốn nhiều thời gian và chi phí.

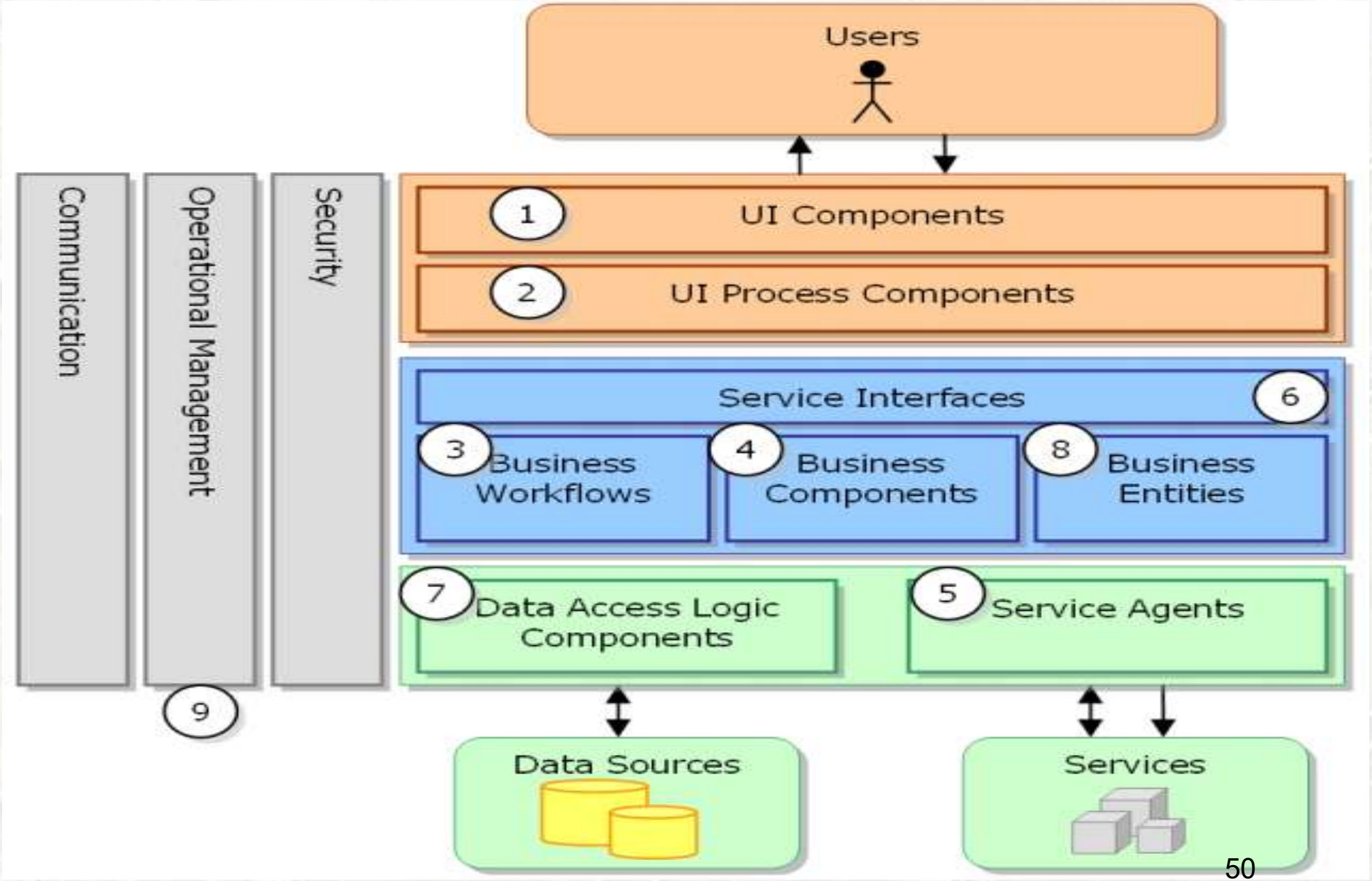
3-tiers

- **Nhược điểm:**
  - Việc truyền dữ liệu giữa các tầng sẽ chậm hơn vì phải truyền giữa các tiến trình khác nhau (IPC)
  - Dữ liệu cần phải được đóng gói → truyền đi → mở gói trước khi có thể dùng được.
  - Việc phát triển ứng dụng phức tạp hơn.

3-tiers

- **Công nghệ:**
  - EJB (J2EE), COM+ (Windows), máy chủ web như nền tảng xây dựng lớp giữa (dùng webservice)
  - máy chủ cơ sở dữ liệu → data tier
- **Ví dụ:**

3-layers


Users
Communication
Operational Management
Security
1
UI Components
2
UI Process Components
Service Interfaces
6
3
Business Workflows
4
Business Components
8
Business Entities
7
Data Access Logic Components
5
Service Agents
9
Data Sources
Services

3-layers

- Phân chia ứng dụng thành các thành phần một cách logic theo chức năng hoặc theo vai trò
- Các layer khác nhau khi được thực thi vẫn có thể nằm trong cùng một vùng bộ nhớ của một process
- Việc giao tiếp giữa 2 layer có thể không phải là giao tiếp giữa 2 process, đồng nghĩa với việc chúng không liên quan tới mô hình client/server.
- Tier liên quan đến cách phân chia một cách vật lý các thành phần trên các máy tính khác nhau.

3-layers

- Tính logic (mỗi layer có 1 công việc) và là 1 thành phần của 3-Tiers.
- Được cấu thành từ: **Presentation Layers**, **Business Layers**, và **Data Layers**.
- Các lớp này sẽ giao tiếp với nhau thông qua các dịch vụ (**Services**) mà mỗi lớp cung cấp để tạo nên ứng dụng
- Lớp này cũng không cần biết bên trong lớp kia làm gì mà chỉ cần biết lớp kia cung cấp dịch vụ gì cho mình và sử dụng nó mà thôi.

3-layers

- Gồm 3 thành phần :
  - **Graphic User Interface (GUI)**: Thành phần giao diện, là các form của chương trình tương tác với người sử dụng.
  - **Business Logic Layer (BLL)**: Xử lý các nghiệp vụ của chương trình như tính toán, xử lý hợp lệ và toàn vẹn về mặt dữ liệu.
  - **Data Access Layer (DAL)**: lớp giao tiếp với các hệ quản trị CSDL.

3-layers

- **Graphic User Interface (GUI)**: Thành phần giao diện
  - UI Components : gồm các thành phần tạo nên giao diện của ứng dụng (GUI). Chúng chịu trách nhiệm thu nhận và hiển thị dữ liệu cho người dùng… Ví dụ : textbox, button, combobox, …
  - UI Process Components : là thành phần chịu trách nhiệm quản lý các quá trình chuyển đổi giữa các UI…
  - Ví dụ : Sắp xếp quá trình kiểm tra thông tin khách hàng:
    - » Hiển thị màn hình tra cứu ID
    - » Hiển thị màn hình thông tin chi tiết khách hàng tương ứng
    - » Hiển thị màn hình liên lạc với khách hàng.

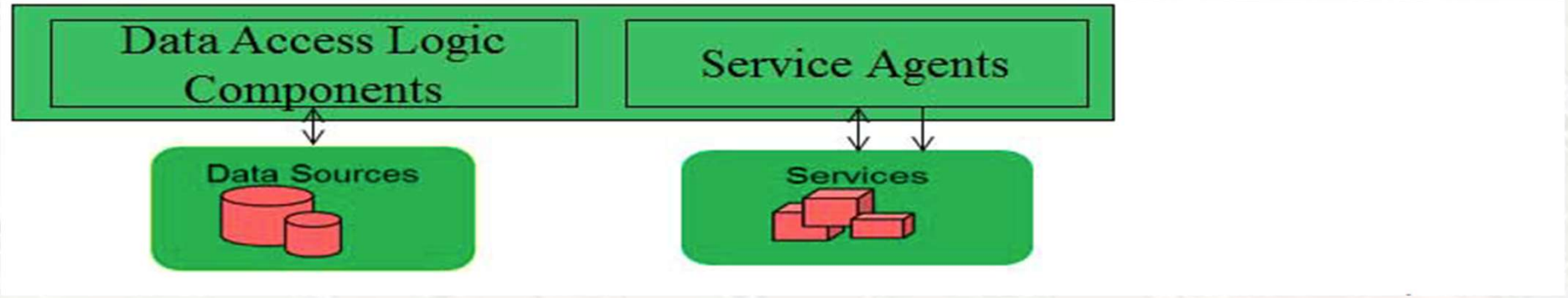
3-layers

- **Bussiness Layer (BLL):** gồm 4 thành phần:

1. Service Interface : là thành phần giao diện lập trình mà lớp này cung cấp cho lớp Presentation sử dụng.
2. Bussiness Workflows: xác định và điều phối các quy trình nghiệp vụ gồm nhiều bước và kéo dài. Những quy trình này phải được sắp xếp và thực hiện theo một thứ tự chính xác.
3. Bussiness Components: kiểm tra các quy tắc nghiệp vụ, ràng buộc logic và thực hiện các công việc . Các thành phần này cũng thực hiện các dịch vụ mà Service Interface cung cấp và Business Workflows sẽ sử dụng nó.
4. Bussiness Entities: Data Transfer Objects (DTO) → truyền dữ liệu giữa các lớp (Presentation và Data Layer) → cấu trúc dữ liệu (DataSets, XML,…) hay các lớp đối tượng đã được tùy chỉnh.

3-layers

- **Data Layer (DAL):**



  - Data Access Logic Components : chịu trách nhiệm chính lưu trữ và truy xuất dữ liệu từ các nguồn dữ liệu (Data Sources) như XML, file system,… .
  - Service Agents: giúp bạn gọi và tương tác với các dịch vụ từ bên ngoài một cách dễ dàng và đơn giản.

3-layers

- **Ví dụ:**
  - Cần một solution riêng cho project.
  - Cần 3 project khác nhau để làm nên 3 lớp, tên Project đặt như sau:
  - Lớp GUI: (VD: QuanLy_GUI)
  - Lớp Business: (VD: QuanLy_BUS) BLL
  - Lớp Data Access: (VD: QuanLy_DAL)
  - Lớp DTO: (VD: QuanLy_DTO) → không cần

3-layers

Ví dụ:

- Để minh họa cho **kiến trúc 3 tầng (3-Tier)** bằng ngôn ngữ Java, người ta thường dùng framework **Spring Boot** với mô hình **3 lớp (3 layers)** phổ biến: Controller (Presentation), Service (Application/Logic), và Repository (Data).
- Dưới đây là cấu trúc thư mục và mã nguồn ví dụ cho chức năng **Quản lý Sản phẩm (Product)**.

3-layers

```
com.example.demo
|
├── controller (Presentation Layer)
|    └── ProductController.java
|
├── service (Business Logic Layer)
|    ├── ProductService.java
|    └── ProductServiceImpl.java
|
├── repository (Data Access Layer)
|    └── ProductRepository.java
|
└── model (Entity)
     └── Product.java
```

3-tiers vs 3-layer

- Cấu trúc phân chia giống nhau (presentation, bussiness , data).
- Hoàn toàn khác nhau.
- Nếu 3 tiers có tính vật lí thì 3 layer có tính logic
- Nghĩa là ta phân chia ứng dụng thành các phần (các lớp) theo chức năng hoặc vai trò một cách logic.
- Các layer khác nhau được thực thi trong 1 phân vùng bộ nhớ của process.
- Một tier có thể có nhiều layer.

3-tiers vs 3-layer

- **Bảo mật cao hơn:** Tầng dữ liệu hoàn toàn bị cô lập. Người dùng từ trình duyệt không thể truy cập trực tiếp vào database, giảm thiểu nguy cơ bị tấn công (như SQL Injection).
- **Dễ bảo trì và nâng cấp:** Do các tầng độc lập, bạn có thể thay đổi toàn bộ giao diện (Front-end) mà không cần viết lại logic hệ thống (Back-end), hoặc đổi từ database này sang database khác mà giao diện không bị ảnh hưởng.
- **Tối ưu hóa hiệu năng:** Bạn có thể đặt mỗi tầng trên các cụm máy chủ (Server) riêng biệt để phân phối tải tốt hơn khi website có lượng truy cập lớn

APPLET

- Khả năng của Applet
  - Applet được đặt tại một Server trên mạng
  - Applet được chuyển tới máy Client theo một trang HTML nào đó
  - Khi một trình duyệt (tương thích với Java) nhận được trang web này, nó sẽ tải mã của Applet và thực thi trên máy client

GV: Võ Tấn Dũng

APPLET

- Giới hạn của Applet
  - Không được nạp các thư viện hay các phương thức sử dụng mã gốc(native code).
  - Không được đọc và ghi lên các tập tin của máy đang chạy chúng.
  - Không được khởi động bất kỳ chương trình nào trên máy đang chạy.
  - Không được đọc bất kỳ tính chất nào của hệ thống

GV: Võ Tấn Dũng

BÀI TẬP

1. Dùng kĩ thuật khung hình phụ để vẽ một quả bóng chuyển động tự do trong một applet.
2. Mở rộng bài toán với 2 quả bóng.
3. Mở rộng bài toán với nền applet là một hình ảnh.
4. Mở rộng bài toán với quả bóng là một hình ảnh và có nhiều quả bóng chuyển động đồng thời.

GV: Võ Tấn Dũng

HẾT
BÀI 5

GV: Võ Tấn Dũng